

This document is the complete, current state of the project. It supersedes any earlier draft — where something changed during design, only the final decision is recorded here. Read this top to bottom and you have everything needed to keep building.

1. The Problem Statement (official, verified)

PS ID: SIH26024 | Theme: Smart Automation | Category: Software | Org: Ministry of Coal (Coal India Limited) Officially tagged technologies (confirmed against the live SIH catalogue, not a paraphrase): Artificial Intelligence (AI), Machine Learning (ML), Blockchain, GIS/Remote Sensing, Mobile App Development, +1 more. Confirmed NOT in scope: IoT / real-time sensor telemetry. That belongs to a neighboring PS (SIH26025 — mine subsidence monitoring), easily confused with ours because it's the same ministry with an adjacent PS number. Do not build IoT for this project under any circumstance.

Background: Coal India's operations span multiple subsidiaries, mine sites, contractors, regulatory bodies, and field offices. Governance activities — statutory compliance monitoring, inspection tracking, safety observations, production reporting, environmental monitoring, worker attendance, contract management, grievance handling, regulatory reporting — are currently managed through fragmented systems, manual documentation, spreadsheets, and delayed reporting. This causes data inconsistency, delayed decisions, limited transparency, compliance gaps, duplicated records, weak field-level monitoring, and no real-time operational insight.

What the solution must do (verbatim from the official brief):

Digitally track statutory compliance for safety, environment, production, and labour regulations. Enable real-time monitoring of inspections, observations, violations, and corrective actions. Use AI/analytics to identify high-risk areas, recurring compliance failures, and operational anomalies. Provide geo-tagged, time-stamped field reporting through mobile apps. Integrate dashboards for mine officials, corporate management, and regulatory authorities. Generate automated alerts, reminders, compliance reports, and escalation mechanisms. Minimize manual paperwork; improve transparency, accountability, decision-making. Be scalable across multiple mines and subsidiaries. Optional tools named explicitly: AI/ML, mobile apps, GIS mapping, OCR/document digitization, workflow automation, blockchain-based audit trails, multilingual conversational interfaces.

Expected Solution (verbatim): a centralized AI-enabled smart governance platform integrating compliance monitoring, inspection management, operational reporting, contractor management, and field activity tracking — with a centralized dashboard, an AI/analytics engine that generates predictive alerts (not just displays logs), a geo-tagged offline-capable mobile app, automated workflows (alerts/reminders/escalations/digital approvals/statutory reports), GIS mapping, OCR digitization, and secure digital audit trails.

The one sentence that matters most for scoring: the brief explicitly wants an AI engine that generates predictive alerts, not one that only reacts after a violation is logged. This is the single most important line in the whole document — see §12.

2. Solution name and one-line pitch

KhanDrishti ("Mine" + "Vision" — Hindi/Sanskrit-rooted, matches the plain-language style of real Ministry of Coal digital initiatives like Khanan Prahari).

One-liner: A shared, role-based governance ledger that turns every inspection, violation, incident, and grievance across CIL's mine sites into one auditable record — scored for risk by a rule engine and a trained ML model working together, automatically escalated against real regulatory deadlines, and never silently editable after the fact.

3. Non-negotiable design principles (violate these and the whole design falls apart) No IoT, no computer vision, no deep learning, anywhere. Confirmed against the official tag list twice. A classifier on structured data is explicitly stated as sufficient in the PS's own feasibility framing. Every category, clause, site, and contractor comes from one shared taxonomy file. The rule engine and the data generator both read from it; neither can invent a category the other doesn't know about. This is what makes drift between rules and data structurally impossible rather than something to remember to check. The rule engine and the ML model are trained/computed independently, then fused. The ML model is never trained to imitate the rule engine's output — it's trained on real (or simulated-real) outcomes, so it can catch what the rule engine structurally cannot see. Every feature fed to a model must respect what kind of data it actually is. Categorical stays categorical (site, contractor, clause — no arbitrary ordering). Ordinal stays ordered (severity — encode the order, don't discard it). Timestamps and coordinates are never fed raw — only derived, causal, relative features (recency, recurrence, trend) ever reach a model. Time-based train/test split, always. Train on chronologically earlier data, test on later. A random split would leak future information backward. Class-weighting, not oversampling, for imbalance. True high-risk cases are a minority; we weight the loss function rather than synthesizing fake examples that might not correspond to real scenarios. Escalation is assigned to a role, resolved against a live duty roster — never to a named person directly. This is the fix for "what if the responsible officer isn't there." Nothing is ever edited in place. Every state change is a new, appended, hash-chained record. There is no UPDATE statement anywhere in the write path for compliance data. Be honest about what's real vs. simulated, always — especially about the satellite/CMSMS integration. We do not have live access to Khanan Prahari/CMSMS. It is a designed future-integration point, demoed as a clearly-labeled mock, never presented as live. Match technique to task — don't reach for something fancier than the data actually needs. Free text gets keyword-nudging and, only in the chat assistant, light retrieval — never a full LLM pipeline in the scoring path. This discipline is a big part of what makes the design defensible.

4. Complete feature list, in priority order Tier 0 — must be 100% working live in the demo, no excuses Offline-first mobile field capture (geo-tag + timestamp auto-captured, local queue, auto-sync on reconnect) Structured inspection/violation logging mapped to exact DGMS/CMR-2017 clause codes (never free-text-only) Rule engine (deterministic, explainable, capped additive scoring) Escalation state machine with SLA deadlines tied to real regulatory response windows, auto-escalation on breach One working role-based dashboard (site / corporate / regulator views) Predictive/forecast output — a site-level forward-looking risk trend, not just per-record scoring. This is the PS's own explicit ask and the single highest-priority gap to close. Tier 1 — build if the team has real bandwidth ML classifier (gradient-boosted trees) fused with the rule engine via noisy-OR SHAP-based explainability surfaced in the UI ("why was this flagged") Peer-benchmarking (percentile risk rank vs. similar mines) Equipment

anomalies as a logged category (just a new clause inside the existing safety pack — cheap) Incidents as a category distinct from violations (a third pluggable rule pack alongside safety and grievance/HR) Distinct verification step (re-inspection confirming a fix actually worked, separate from closure sign-off) Lightweight keyword auto-suggest for clause selection from free-text description (human still confirms) GIS map view (visual pins on a map, not just stored coordinates) Multilingual (Hindi/English) chat assistant for natural-language queries over the ledger Tier 2 — design it, explain it, mock/simulate it in the demo — don't try to fully build OCR ingestion for scanned paper registers (with fuzzy-matching against the taxonomy's clause codes) Auto-monitor: missed-report detection, CUSUM drift check, satellite cross-check (clearly labeled as simulated, see §9) Interoperability API (sync design with Khanan Prahari/CMSMS and CIL's own ICIS — described, not live-connected) Contractor appeal portal Public aggregate compliance scorecard (RTI/Parliament-answer-ready format) Explicitly excluded — do not build, do not demo as if built IoT / real-time sensor telemetry Computer vision on inspection photos Any deep learning model A real production Hyperledger blockchain deployment (we use hash-chaining instead — same tamper-evidence property, honestly buildable) Voice input (not in the official PS text; a nice-to-have someone else suggested, not a scope gap) AI auto-classification of an issue directly from a photo (would require computer vision — ruled out on principle) Zone/panel-level risk granularity (PS talks about mines and subsidiaries as the scaling unit, never sub-zones) Building attendance, production reporting, and contract management as full separate systems — mention as roadmap, prove generalization with one extra vertical (grievance/incidents) through the same pipeline instead

5. System architecture — complete, tier by tier

(This is the final, v4 architecture. Eight tiers, top to bottom.)

Tier 1 — Client & Access (4 entry points) Mobile app — "Offline + geo-tag." Field inspector's primary tool. Captures location/timestamp automatically, queues locally when offline, syncs automatically on reconnect. Web dashboard — "Role views + map." Site officials, corporate management, and regulators each see a scoped view of the same data. Includes the GIS map layer (Tier 1 additions). Chat assistant — "Hindi/English query." Natural-language search over the ledger and free-text descriptions ("show me sites with recurring ventilation issues last quarter"). The one legitimate home for heavier NLP in this system, because the PS explicitly asks for it. External portals — "Appeal & scorecard." Two distinct audiences bundled visually: an authenticated contractor self-view + formal appeal form, and a public, anonymized, read-only aggregate scorecard. Tier 2 — Ingestion (3 paths into the pipeline) OCR ingestion — "Digitizes paper logs." An inspector photographs an existing paper register; OCR extracts fields; those fields are fuzzy-matched against the shared taxonomy's exact clause codes before entering the same pipeline as a manual entry. The matching step, not the OCR accuracy, is the real risk here. API gateway — "Auth, RBAC, i18n." Single front door. Confirms identity and permissions, acknowledges receipt immediately (doesn't wait for scoring to finish — this is what keeps the mobile app responsive even when many offline reports sync at once). Auto-monitor — "Silence/drift check." The one component with no incoming arrow from Tier 1 — it runs on its own schedule, not triggered by a human. Three jobs: (a) turns an overdue expected inspection into a violation record in its own right (missed reporting is already an offense under real DGMS regulation, this just enforces it automatically); (b) a CUSUM statistical check for a site whose reporting rate is slowly fading rather than stopping outright; (c) cross-checks against Khanan Prahari/CMSMS satellite data for activity with no matching inspection record — see §9 for how this is honestly framed. Tier 3 — Job queue Job queue — "Dispatch, features, suggest." All three ingestion paths converge here. The gateway has

already replied to the client, so this is where the real work happens without making anyone wait: feature engineering (turning a raw record into recurrence counts, cross-site spread, site violation rate) and the lightweight keyword scan that suggests a clause code and nudges severity from alarm words in the free-text description. Tier 4 — Intelligence (only two of these four are genuinely AI — see §6) Rule engine — "Safety, HR, incidents." Deterministic, explainable, capped additive formula. "Packs" is the key word: the same engine loads a safety (DGMS/CMR-2017) rule set, an HR/grievance rule set, or an incidents rule set interchangeably — proving the platform generalizes across governance domains without a rebuild. ML classifier — "Gradient-boost score." The actual AI. A trained model outputs a probability that this specific combination of factors is genuinely high-risk, including patterns the rule engine structurally cannot see. Peer benchmark — "Percentile vs peers." Statistics, not a trained model — ranks a site/contractor against similar peers using a percentile (not a z-score, because violation counts are right-skewed and a percentile makes no assumption about distribution shape). Explainability — "SHAP attribution." Genuinely AI-adjacent — breaks the classifier's score into which factors drove it, in plain language. TreeSHAP specifically, because it's exact and fast for tree models (unlike LIME, which can vary between runs on the same input). Tier 5 — Unified priority score

Fuses the rule score and ML probability via noisy-OR ($\text{combined} = R + M - (R \times M) / 100$), not a simple average — deliberately biased toward flagging if either method raises a concern, because missing a real high-risk case costs more than one extra false alarm.

Tier 6 — Escalation engine Escalation engine — "State machine, SLA, verify." Lifecycle: open → assigned → in-progress → escalated → closed → verify (the new, distinct re-inspection step, separate from the closure sign-off itself). Every stage carries a deadline tied to the real regulatory response window. Role-based, not person-based: assigned to a duty role, re-resolved against a live roster at assignment and at every SLA check, so one person's absence can't silently stall an escalation. Tier 7 — Data & Interoperability Hash-chained ledger — "Tamper-evident by design." Every action written once; each entry stores a hash of the entry before it, so altering any past record breaks every hash after it. This is the concrete, honestly-buildable answer to Blockchain being an officially tagged technology for this PS. Interoperability API — "Syncs CMSMS & CIL ICIS." Designed to connect with what already exists (CIL's own contractor/worker registry, the Ministry's satellite system) instead of building a fourth silo. Tier 8 — Predictive forecast (new, closes the #1 gap) Predictive forecast — "30-day site risk trend." Reads backward through the ledger's history for a site (not a single record's features, the way the other Intelligence boxes do) and produces a forward-looking trend. This is the direct, literal answer to the PS's own line: "generate predictive alerts." Feeds back into the Web dashboard's map view as the site risk trend (not drawn as a line on the diagram, to avoid crossing the whole chart, but real in the design). 6. Where the AI actually is — say this precisely, every time

Only two boxes in the entire architecture are genuinely AI: the ML classifier and Explainability. Everything else is either plain automation (escalation clock, routing, ledger writing — reliable, not intelligent) or statistics (rule engine's arithmetic, peer benchmark's percentile ranking — real math, not a trained model). Being able to point at exactly two boxes when a judge asks "where's the AI" is a stronger answer than gesturing at the whole system.

Automation vs. AI, in one line: automation handles the repetitive, rule-following parts (routing, deadlines, record-keeping) so people stop doing that by hand. AI handles the parts that need judgment — spotting a

risky pattern spread across sites that no fixed rule was written for — and shows its reasoning instead of hiding it.

7. AI/ML — the technical decisions, and why each one over the alternatives

Algorithm: Gradient-boosted decision trees (HistGradientBoostingClassifier in the actual code).

vs. Logistic regression: risk here is interaction-driven (severity and repeat and cross-site spread, together, is worse than the sum of parts); trees discover interactions automatically, logistic regression needs them hand-crafted in advance. vs. Random Forest: both are legitimate per the published mining-safety ML literature; gradient boosting's sequential error-correction typically edges out slightly better accuracy on tabular data of this kind. Defensible honest answer to a judge: "primary choice, would benchmark against Random Forest given more real data." vs. deep learning: ruled out — small data regime is exactly where tree ensembles beat neural nets on tabular data, and it's explicitly against the PS's own feasibility framing. vs. SVM: no natural calibrated probabilities, no exact fast explainability method the way trees have with TreeSHAP.

Explainability: SHAP (TreeSHAP), not LIME or plain feature importance.

Plain feature importance only describes the model in general, not one specific decision. LIME can give different explanations on repeated runs for the same input — bad when a regulator might ask twice. TreeSHAP is exact and fast specifically for tree models — the choice of model and the choice of explainability method reinforce each other by design.

Fusion formula: noisy-OR, $P_{\text{final}} = R + M - (R \times M)/100$ — not a simple average. An average would let a genuinely dangerous pattern get diluted just because the rule engine didn't happen to flag it; noisy-OR keeps the combined score at least as confident as whichever method is more concerned.

Peer benchmarking: percentile rank (empirical CDF), not z-score. Violation counts are right-skewed (most contractors have very few, a small number have many); a z-score assumes something close to a bell curve and would distort the comparison. Percentile rank makes no assumption about distribution shape.

Train/test split: time-based (train on earlier months, test on later), not random. A random split leaks a contractor's later behavior backward into training data used to "predict" their earlier record.

Class imbalance: sample-weighting in the training loss, not SMOTE/oversampling. Keeps every training example grounded in something real rather than synthesizing feature combinations that might not correspond to physically real scenarios.

Evaluation metric: recall weighted higher than precision, and threshold-independent metrics (ROC-AUC, PR-AUC) reported alongside any fixed-threshold number. Missing a real high-risk case costs more than one extra false alarm a human has to dismiss — this is an explicit, statable design choice, not a default.

Real results actually obtained (from a genuinely trained model on the synthetic dataset, not fabricated numbers):

Method	ROC-AUC	PR-AUC	Recall @ threshold 50
Rule engine alone	0.625	0.491	82.4%
ML classifier alone	0.676	0.596	29.4%
Combined (noisy-OR)	0.677	0.602	94.1%

Both AUC metrics show the ML model genuinely adds discriminative power beyond the rule engine; the combined approach hits the explicit design goal of maximizing recall. This is a small dataset (298 synthetic violations) — expect these numbers to sharpen as real closed-violation outcomes accumulate.

8. Data strategy

One file (taxonomy.yaml) is the only place violation categories, clause IDs, severities, sites, and contractors are defined. The rule engine's conditions and the data generator's outputs both read from it exclusively — neither can reference something the other doesn't know about. This is what makes drift structurally impossible rather than something to manually audit.

What's actually in the taxonomy (real-world grounded, verified) Safety domain clauses: CMR 2017 Reg 106 (ventilation), Reg 108 (roof/side support), Reg 118 (illumination), Reg 154 (explosives), OSH&W Code 2020 Sec 12 (PPE) — each with a `base_risk_multiplier` representing real-world severity of consequence (roof support and explosives weighted highest — roof falls are the largest cause of coal mine fatalities). Grievance/HR domain clauses: wage payment delay, unsafe working hours reported — proves the rule engine isn't safety-only. Severities: Minor/Major/Critical, each with a weight (20/50/80) and a real SLA deadline (7 days / 72 hours / 24 hours respectively). Sites — real, verified mines, not invented: Jharia Colliery, BCCL, underground, Dhanbad district, Jharkhand — India's principal coking coal source, known for underground fires since 1916. Gevra OC & Kusmunda OC, SECL, opencast, Korba district, Chhattisgarh — together over 100 million tonnes/year, among the world's largest opencast coal mines. Real PARIVESH environmental clearance filings show Kusmunda's lease area at 2,991.9 hectares. Dipka OCP, SECL, opencast, Katghora Tehsil, Korba district — real lease area 1,999.3 hectares per PARIVESH. Sonapur Bazari OCP, ECL, opencast, Paschim Bardhaman district, West Bengal (Raniganj Coalfield). Nigahi OCP, NCL, opencast, Singrauli district, Madhya Pradesh — real PARIVESH filing shows expansion from 22.50 to 25.00 MTPA, lease area 3,018.4 to 3,582.7 hectares. Contractors: representative names (Shree Mining Works, Bharat Excavators, Coalfield Logistics Pvt Ltd, CIL departmental crew) — not real named contractors, since that data isn't public. What real public data exists, and at what resolution — be precise about this with judges PARIVESH portal (environment clearance filings) — genuinely mine-specific: exact lease area, production capacity, district/village/state. Best source for realistic site master data. DGMS annual reports / accident-statistics portal — subsidiary-wise and state-wise aggregate accident statistics only (e.g., 2006 report: roof-fall accidents 21% SECL, 18% ECL, 15% BCCL...). The DGMS portal has a "mine user login" — meaning individual mines do report incident data into a real system, but it's gated, not publicly queryable. data.gov.in / Open Government Data Platform — state-wise coal mine accident counts, aggregate only. CIL/Ministry of Coal annual reports — subsidiary-level workforce counts (e.g., SECL ~41,832 employees) — subsidiary total, never per individual mine. What does not exist publicly, confirmed by all of the above: a per-mine, per-date violation/incident log ("Mine X had violation Y on date Z"). This is exactly why the dataset is synthetic — grounded in real regulation structure and real aggregate proportions, not real per-record data, because that data lives inside CIL's internal systems and isn't public. How the synthetic dataset was actually built Generate a raw stream of inspection events in chronological order — most compliant, a minority genuinely violations (roughly 74%/26% split) — because an all-violation dataset teaches a classifier nothing (no negative examples). For every violation, compute features causally — only ever looking backward from that event's own date, never forward (no look-ahead leakage). Compute the rule engine's score using the same simple, capped, additive formula literally used in the app. Simulate a "true" latent risk using a deliberately different, richer formula — including a severity×recurrence interaction term

and per-clause/mine-type effects the simple rule engine cannot see — then sample a binary outcome from it. This is what gives the ML model something genuine to discover; it's never trained to imitate the rule engine. Recalibrated so the true positive rate lands around 27.5% (a realistic minority), not the ~61% the first uncalibrated pass produced. Non-numeric data — how each type is actually handled Categorical (site, contractor, clause, mine type): native categorical handling in the gradient-boosting model, not label-encoding as arbitrary integers — avoids implying a false order between categories that have none. Ordinal (severity): mapped to weights (20/50/80) that encode a real judgment about the size of each jump, not just an arbitrary 1/2/3. Free text (inspector's description): currently only used for keyword-based severity nudging and clause auto-suggestion — deliberately not fed into the risk model directly; genuine NLP is reserved for the chat assistant only, where the PS actually asks for it. Images (inspection photos): stored as evidentiary documentation for human review, never analyzed by a model — computer vision is explicitly out of scope. Geospatial (lat/long): never fed raw into the risk model (would falsely imply nearby points are similar-risk); reserved for the GIS map visualization only. `site_id` is the model's proxy for location. Timestamps: never fed raw; only derived relative/causal features (recurrence count, days-since-last, trailing-window rate) ever reach a model. Graph-shaped data (contractor-across-many-sites): summarized into a single derived number (`cross_site_count`) rather than reaching for a graph neural network — matches the technique to the actual decision needed, not to how sophisticated the underlying structure looks. 9. The "what if nobody reports" layer — and how to talk about it honestly

Every input except one depends on a human deciding to report something. The Auto-monitor (Tier 2) is the fix:

Missed-report detection: DGMS regulation already mandates an inspection cadence; if the expected report doesn't arrive in its window, the system generates its own violation record for the failure to report itself — enforcing an existing rule, not inventing a new one. CUSUM drift check: catches a site's reporting rate slowly fading over time (a cumulative-sum control chart), not just a hard stop — chosen because it needs no training data, works with very little data per site, and is explainable in one sentence, unlike a heavier anomaly-detection model that would be solving a harder problem than the one that actually exists. Satellite cross-check (Khanan Prahari/CMSMS): be extremely careful how this is presented. We do not have live access to this government system. Never say "our solution uses satellite data" as a present-tense fact. Always frame it as: designed for future integration, since it's an existing government-owned system we'd need permission to connect to — not something we'd build ourselves — and demoed here as a clearly-labeled simulation, not a live connection. This honest framing is a stronger answer in front of judges than pretending it's live, because no one expects a student team to have real government API access in 36 hours; what they're evaluating is whether you understand the difference between what's built and what's proposed. Role-based, roster-resolved escalation: fixes the "responsible officer isn't there" problem — assignment is to a duty role, re-resolved live, never to a fixed named person.